



Институт автоматизации и информационных технологий

Кафедра "Программная инженерия"

СОГЛАСОВАНО

ЗАВ. КАФЕДРОЙ

_____ **Абдолдина Ф.Н.**

«__» _____ **2024 г.**

УТВЕРЖДЕНО

ДИРЕКТОР ИАИТТ

_____ **Ускенбаева Р.К**

«__» _____ **2024 г.**

[Ka]

Дипломдық жобаға
ТҮСІНДІРМЕ ЖАЗБА

6B06102 — Computer Science білім беру бағдарламасы

Алматы 2026

ҚАЗАҚСТАН РЕСПУБЛИКАСЫ
ҒЫЛЫМ ЖӘНЕ ЖОҒАРЫ БІЛІМ МИНИСТРЛІГІ

Қ.И. Сәтбаев атындағы Қазақ ұлттық техникалық зерттеу университеті

Автоматика және ақпараттық технологиялар институты

«Программалық инженерия» кафедрасы

ҚОРҒАУҒА ЖІБЕРІЛДІ
ПИ кафедрасының меңгерушісі
техн. ғыл. канд., қауымд.проф.
_____ Ф.Н. Абдолдина
« ____ » _____ 2026 ж.

Дипломдық жобаға
ТҮСІНДІРМЕ ЖАЗБА

Тақырыбы: «Маркетплейс-пакшоттарды автоматты түрде құруға арналған
AI-кескін редакторын әзірлеу»

6B06102 — Computer Science білім беру бағдарламасы

Орындады

[Аты-жөні]

Ғылыми жетекшісі

техн. ғыл. канд., қауымд.проф.

_____ [Аты-жөні]

" ____ " _____ 2026 ж.

Рецензент

техн. ғыл. канд., доцент

_____ [Аты-жөні]

" ____ " _____ 2026 ж.

Алматы 2026

АҢДАТПА

Бұл дипломдық жоба өнімді фотосуреттерді автоматты түрде өңдеуге арналған толық стектік веб-қосымшаны әзірлеуге арналған. Жобада фон жою үшін BiRefNet, кескінді 4 есе ұлғайту үшін AuraSR-v2, нүкте бойынша объектіні бөлу үшін MobileSAM, артық объектілерді жою үшін LaMa, беттерді қалпына келтіру үшін GFPGAN және төмен жарықтандырылған фотосуреттерді жөндеу үшін Zero-DCE сияқты заманауи нейрондық желілер модельдері пайдаланылады. Бұл жоба маркетплейс-пакшоттарды автоматты түрде құруға арналған конвейерді қамтиды: QC дарбазалары арқылы фон жою → AI ұлғайту → ақ canvas-қа композиция → Auto-Fix циклі.

Backend бөлігі FastAPI және PyTorch CUDA 11.8 пайдалана отырып, GTX 1650 4GB VRAM шектеуіне оңтайландырылған. Жүйеде LRU eviction арқылы lazy loading модельдер, нәтижелерді кәштеу, SSE арқылы нақты уақыттағы прогресс және Prometheus/Grafana арқылы мониторинг қолданылған. Frontend Next.js 14, TypeScript және Fabric.js негізінде жасалған.

АННОТАЦИЯ

Данный дипломный проект посвящён разработке полнофункционального веб-приложения для автоматизированной обработки продуктовых фотографий. В проекте применяются современные модели нейронных сетей: BiRefNet для удаления фона, AuraSR-v2 для четырёхкратного увеличения изображений, MobileSAM для точечной сегментации объектов, LaMa для удаления нежелательных объектов, GFPGAN для восстановления лиц и Zero-DCE для коррекции слабоосвещённых снимков. Реализован конвейер автоматической генерации маркетплейс-пакшотов: удаление фона → AI-апскейл → композиция на белом канвасе → QC-гейты → цикл Auto-Fix.

Backend построен на FastAPI и PyTorch CUDA 11.8 и оптимизирован под аппаратное ограничение GTX 1650 4GB VRAM. В системе применены ленивая загрузка моделей с LRU-эвикцией, многоуровневое кэширование, потоковая передача прогресса через SSE и мониторинг через Prometheus/Grafana. Frontend разработан на Next.js 14, TypeScript и Fabric.js. Покрытие тестами — 227 модульных и интеграционных тестов pytest, развёрнут CI/CD через GitHub Actions.

ABSTRACT

This diploma project is dedicated to the development of a full-stack web application for automated product photography processing. The project employs modern neural network models: BiRefNet for background removal, AuraSR-v2 for 4× image upscaling, MobileSAM for point-prompted object segmentation, LaMa for unwanted object inpainting, GFPGAN for face restoration, and Zero-DCE for low-light photo correction. An automatic marketplace packshot pipeline is implemented: background removal → AI upscale → composition on a white canvas → QC gates → Auto-Fix loop.

The backend is built on FastAPI and PyTorch CUDA 11.8, optimized for the hardware constraints of an NVIDIA GTX 1650 with 4GB VRAM. The system features lazy model loading with LRU eviction, multi-layer caching, real-time progress streaming via SSE, and observability through Prometheus/Grafana. The frontend is implemented in Next.js 14, TypeScript, and Fabric.js. Test coverage includes 227 unit and integration tests with pytest, and a CI/CD pipeline is deployed via GitHub Actions.

СОДЕРЖАНИЕ

Введение	8
1 Научно-исследовательский раздел	10
1.1 Цель разработки системы	10
1.2 Определения, термины и сокращения	11
1.3 Состояние вопроса и область применения	12
1.4 Описание предметной области	14
1.5 Обзор существующих решений	16
1.6 Обоснование выбора моделей и технологий	19
2 Проектная часть	22
2.1 Функциональные требования	22
2.2 Нефункциональные требования	24
2.3 Архитектура системы	26
2.4 Проектирование REST API	28
2.5 Пользовательские интерфейсы	30
3 Экспериментальная часть	32
3.1 Реализация backend на FastAPI	32
3.2 Ленивая загрузка моделей и LRU-эвикация	34
3.3 GPU-сериализация и контроль конкурентности	36
3.4 Многоуровневое кэширование	38
3.5 Реализация конвейера пакшота с QC-гейтами	40
3.6 Поточковая передача прогресса через SSE	43
3.7 Реализация frontend на Next.js	45
3.8 Тестирование (pytest, 227 тестов)	48

3.9 CI/CD через GitHub Actions	50
3.10 Мониторинг через Prometheus и Grafana	52
3.11 Замеры производительности и оптимизация	55
Заключение	58
Список использованных источников	60
Приложение А — Техническое задание	62
Приложение Б — Листинг ключевых модулей	65

ВВЕДЕНИЕ

Карточка товара на маркетплейсе начинается с фотографии, и именно она формирует первое решение покупателя — открыть карточку или пролистать дальше. Площадки Wildberries, Ozon, Kaspi и Amazon закрепляют это в технических регламентах: снимок обязан иметь однородный (чаще белый) фон, фиксированную долю объекта в кадре, резкие границы без ореолов и полное отсутствие посторонних предметов. Приведение произвольной фотографии к этим требованиям вручную — в Adobe Photoshop или GIMP — отнимает от 5 до 15 минут даже у подготовленного ретушёра. Для продавца с ассортиментом в сотни позиций ручная ретушь превращается в постоянную статью расходов и сдерживает скорость вывода товаров на витрину.

Актуальность темы определяется разрывом между зрелостью отдельных нейросетевых моделей и отсутствием доступного инструмента, который объединял бы их в сквозной процесс. Сегментация (BiRefNet, RMBG-2.0), суперразрешение (Real-ESRGAN, AuraSR), инпейнтинг (LaMa) и реставрация лиц (GFPGAN) по отдельности уже дают результат на уровне ручной ретуши. Однако их сборка в единый конвейер, дополненный автоматической проверкой качества выхода и при этом помещающийся в 4 ГБ видеопамати потребительской видеокарты, остаётся самостоятельной инженерной задачей — именно ей и посвящена настоящая работа.

Объект исследования — процесс автоматизации обработки продуктовых фотографий для маркетплейсов с использованием современных моделей машинного обучения.

Предмет исследования — программный комплекс, реализующий конвейер удаления фона, повышения разрешения и композиции изображений на едином канвасе с автоматическим контролем качества.

Цель дипломного проекта — разработать полнофункциональное веб-приложение, которое автоматизирует процесс подготовки маркетплейс-пакшотов из произвольных пользовательских фотографий, обеспечивая качество, сопоставимое с ручной обработкой, при существенно меньших временных затратах.

В соответствии с поставленной целью определены следующие задачи:

- Провести анализ существующих решений (Potoroom, remove.bg, Cleanup.pictures) и выявить их сильные и слабые стороны.

- Выбрать архитектуру нейронных сетей, оптимальную по соотношению качество/скорость на потребительском GPU (NVIDIA GTX 1650, 4GB VRAM).
- Спроектировать backend на FastAPI с REST API, поддерживающий ленивую загрузку моделей, кэширование результатов и сериализацию GPU-операций.
- Реализовать конвейер автоматической генерации пакшотов с QC-гейтами (контроль чистоты фона, отсутствие halo-артефактов, целостность маски).
- Разработать frontend на Next.js с интерактивным редактором (Fabric.js), панелью AI-инструментов и пакетной обработкой.
- Покрыть код модульными и интеграционными тестами ($\geq 80\%$ покрытия на критических модулях).
- Развернуть систему в Docker, настроить CI/CD и мониторинг (Prometheus + Grafana).

Практическая значимость заключается в том, что система снимает зависимость небольшого e-commerce-продавца от услуг ретушёра: подготовка одного пакшота сокращается с минут до единиц секунд, а пакетный режим обрабатывает десятки изображений за один прогон. Так как развёртывание локальное, фотографии не покидают инфраструктуру пользователя и не загружаются в сторонние облачные сервисы.

Структура работы. Пояснительная записка включает введение, три основных раздела — научно-исследовательский, проектный и экспериментальный, — заключение, список использованных источников и два приложения. В первом разделе анализируется предметная область и обосновывается выбор моделей и технологий; во втором формализуются требования, проектируются архитектура и REST API; в третьем описываются реализация подсистем, тестирование и измеренные характеристики производительности.

1 Научно-исследовательский раздел

1.1 Цель разработки системы

Разработать веб-приложение для автоматизированной обработки продуктовых фотографий с использованием современных нейронных сетей, удовлетворяющее следующим характеристикам:

- Универсальность входных данных: приём фотографий с произвольным фоном, разрешением от 800 до 6000 пикселей по длинной стороне, в форматах JPEG, PNG, WebP.
- Качество выходных данных: удаление фона с точностью пиксельной границы, размер канваса 1920×1920 или 2560×2560 пикселей, белый или прозрачный фон, центрированный объект с заданным отступом.
- Производительность: обработка одного изображения за 3–10 секунд на NVIDIA GTX 1650 (4GB VRAM), пакетная обработка 50 изображений за 5–8 минут.
- Доступность: работа через браузер без установки клиентских приложений, поддержка десктопных и мобильных устройств.
- Расширяемость: возможность подключения дополнительных AI-операций (восстановление лиц, цветокоррекция, инпейнтинг) к единому конвейеру.

1.2 Определения, термины и сокращения

В таблице 1.1 приведены сокращения и термины, используемые далее в работе.

Таблица 1.1 — Список сокращений и терминов

Сокращение / термин	Расшифровка
AI	Artificial Intelligence (искусственный интеллект)
API	Application Programming Interface (программный интерфейс)
BiRefNet	Bilateral Reference Network — архитектура для high-resolution салиент-сегментации
CUDA	Compute Unified Device Architecture (NVIDIA)
FastAPI	Python-фреймворк для построения REST API
GFPGAN	Generative Facial Prior GAN — модель реставрации лиц

HF	Hugging Face — платформа хостинга ML-моделей
LRU	Least Recently Used (стратегия вытеснения кэша)
LaMa	Large Mask Inpainting — модель закрашивания удалённых объектов
ML	Machine Learning
ONNX	Open Neural Network Exchange — формат для портативного inference
Packshot	Студийная фотография товара на нейтральном фоне (обычно белом)
QC	Quality Control (контроль качества)
REST	Representational State Transfer (архитектурный стиль API)
RMBG	Remove Background — серия моделей сегментации от BriaAI
SAM	Segment Anything Model (Meta)
SSE	Server-Sent Events — механизм потоковой передачи от сервера
TTL	Time To Live (время жизни записи в кэше)
VRAM	Video RAM (видеопамять)
Zero-DCE	Zero-reference Deep Curve Estimation — модель коррекции освещения

1.3 Состояние вопроса и область применения

Центральная операция системы — отделение объекта от фона — в компьютерном зрении формулируется как задача *salient object segmentation*: требуется отнести каждый пиксель снимка либо к значимому объекту, либо к фону. Сложность задачи скрыта в граничных областях — волосах, мехе, полупрозрачных и сетчатых элементах, где «правильного» бинарного ответа по сути не существует, и качество модели определяется именно тем, насколько аккуратно она обрабатывает эти зоны.

За последнее десятилетие подходы к решению прошли несколько поколений, и для обоснования выбора модели важно понимать, чем каждое из них ограничено:

- Пороговые и контурные методы (до 2015 г.) — GrabCut, водораздел, активные контуры. Опираются на ручные подсказки оператора и яркостные

перепады, из-за чего на тонких границах дают рваную маску. Для автоматического конвейера непригодны.

- Энкодер-декодеры с пропускающими связями (2015–2019 гг.) — семейство U-Net, включая U2Net и облегчённый U2NetP. Первыми обеспечили приемлемое качество без пользовательского ввода и долго оставались отраслевым стандартом.
- ISNet (2022 г.) — переход к промежуточному обучению по контуру; легла в основу коммерческой RMBG-1.4 от BriaAI.
- BiRefNet (2024 г.) — двусторонняя референсная схема (bilateral reference), сочетающая глобальный контекст сцены с локальной детализацией краёв. На сегодня лучшее качество в открытом доступе; на этой же архитектуре построена RMBG-2.0.

Параллельно развивалось суперразрешение. Свёрточные пионеры SRCNN и FSRCNN (2014–2016 гг.) восстанавливали лишь общие контуры; переход к состязательному обучению в ESRGAN и Real-ESRGAN (2018–2021 гг.) позволил «дорисовывать» правдоподобную мелкую текстуру; наконец, AuraSR-v2 (2024 г.) на базе GigaGAN заметно опережает Real-ESRGAN по перцептивному качеству, что и определило его выбор в данной работе.

Хотя система проектировалась под маркетплейс-фотографию, её конвейер применим шире: помимо подготовки карточек товара, это единообразное оформление постов для SMM, удаление личных вещей с интерьерных снимков в каталогах недвижимости и предобработка изображений для последующих систем компьютерного зрения.

1.4 Описание предметной области

Прежде чем автоматизировать подготовку пакшота, требования маркетплейсов нужно перевести из текста регламентов в измеримые числовые параметры — только так их можно заложить в программные проверки. Обобщённые требования площадок сведены в таблице 1.2; именно эти величины далее становятся порогами QC-гейтов (раздел 3.5).

Таблица 1.2 — Требования к маркетплейс-фотографиям

Параметр	Требование (типичное)
Разрешение	от 900×1200 до 4500×4500 px
Фон	белый ($RGB \geq 248, 248, 248$) или прозрачный

Доля объекта	70–85% от площади канваса
Отступы от краёв	7.5–10% от размера канваса
Формат	JPEG (≤ 5 МБ) или PNG
Соотношение сторон	3:4, 1:1, 4:5
Тени	мягкие контактные допустимы
Артефакты	без обрезаний, halo, цветовых пятен

Технологически процесс автоматизированной подготовки пакшота состоит из последовательности шагов:

- Приём и валидация входного изображения.
- Анализ свойств (яркость, разрешение, ориентация).
- Условная цветокоррекция (для тёмных фото).
- Условное AI-увеличение (для малых разрешений).
- Семантическая сегментация — выделение объекта.
- Постобработка маски (морфологическая фильтрация, удаление мелких компонент).
- Композиция на чистом канвасе с заданными отступами.
- Контроль качества — серия гейтов проверки.
- Автоматическое исправление обнаруженных проблем (Auto-Fix).
- Финальный рендеринг в PNG.

1.5 Обзор существующих решений

На рынке представлены как узкоспециализированные сервисы (решающие одну задачу — например, только удаление фона), так и комбайны с подпиской. Чтобы определить незанятую нишу, проведено сопоставление четырёх показательных продуктов — Potoroom, remove.bg, Cleanup.pictures — с разрабатываемой системой NovaEdit по набору функциональных и эксплуатационных критериев (таблица 1.3).

Таблица 1.3 — Сравнение существующих решений

Критерий	Photoroom	remove.bg	Cleanup	NovaEdit
Удаление фона	✓	✓	✓	✓ (BiRefNet)
Апскейл	✓ (платно)	✗	✗	✓ (AuraSR)
Инпейнтинг	✗	✗	✓	✓ (LaMa)
Реставрация лиц	✗	✗	✗	✓ (GFPGAN)
Цветокоррекция	✓ (Pro)	✗	✗	✓ (Zero-DCE)
Пакетная обработка	✓ (платно)	✓ (API)	✗	✓ (бесплатно)
QC-гейты	✗	✗	✗	✓
Auto-Fix	✗	✗	✗	✓
Локально	✗	✗	✗	✓
Open source	✗	✗	✗	✓
Цена/мес	\$9.99	\$9	\$5	бесплатно

Ключевые преимущества разработанной системы:

- Локальное развёртывание — данные не покидают инфраструктуру пользователя.
- Полный pipeline в одном продукте, включая редкие операции (инпейнтинг, реставрация лиц).
- QC-гейты с Auto-Fix — отсутствуют у конкурентов.
- Прозрачность и расширяемость — открытый код, возможность дообучения моделей.

1.6 Обоснование выбора моделей и технологий

1.6.1 Выбор моделей нейронных сетей

Удаление фона: BiRefNet (ZhengPeng7/BiRefNet)

Критерии: качество масок на сложных границах, открытая лицензия, размер модели в пределах 1 ГБ.

Для оценки качества сегментации используется метрика Intersection over Union (IoU), формализующая степень совпадения предсказанной маски A с эталоном B на множестве пикселей:

$$IoU(A, B) = |A \cap B| / |A \cup B| \quad (1.1)$$

где $|\cdot|$ обозначает количество пикселей. Значение $IoU = 1$ соответствует идеальному совпадению, $IoU = 0$ — полному отсутствию пересечения. Для оценки модели на наборе из N изображений берётся усреднённое значение — mean IoU:

$$mIoU = (1/N) \cdot \sum IoU(A_i, B_i), \quad i = 1..N \quad (1.2)$$

Сравнение моделей по mIoU на бенчмарке DIS-5K (Dichotomous Image Segmentation) приведено в таблице 1.4.

Таблица 1.4 — Сравнение моделей сегментации

Модель	Параметры	Размер	mIoU (DIS-5K)	Лицензия
U2Net	4.7M	177 МБ	0.872	Apache 2.0
RMBG-1.4	4.7M	177 МБ	0.911	RAIL-M
RMBG-2.0	220M	885 МБ	0.962	gated
BiRefNet	220M	885 МБ	0.960	MIT

Выбрана BiRefNet — даёт качество, сопоставимое с RMBG-2.0, но имеет полностью открытую MIT-лицензию, что критично для коммерческого использования.

Апскейл: AuraSR-v2

Для суперразрешения качество удобнее измерять не попиксельным PSNR, а перцептивной метрикой LPIPS (Learned Perceptual Image Patch Similarity), которая сравнивает изображения в пространстве признаков предобученной сети и лучше согласуется с восприятием человека; меньшее значение соответствует большему сходству с эталоном. Сравнение моделей суперразрешения приведено в таблице 1.5.

Таблица 1.5 — Сравнение моделей суперразрешения

Модель	Параметры	LPIPS (↓)	VRAM 1024px
Real-ESRGAN	17M	0.176	1.2 ГБ
AuraSR-v2	57M	0.143	1.5 ГБ

StableDiffusion x4	700M	0.155	4+ ГБ
--------------------	------	-------	-------

AuraSR-v2 даёт лучшее визуальное качество при умеренном расходе VRAM, что важно для GTX 1650 (4ГБ). Для безопасной работы используется метод `upscale_4x_overlapped(max_batch_size=1)`.

Остальные модели:

- MobileSAM (для object cutout по точечным подсказкам) — компактная версия SAM от Meta, всего 40 МБ, работает на CPU за приемлемое время.
- LaMa (для инпейнтинга) — стандарт de-facto для закрашивания удалённых областей.
- GFPGAN v1.4 (для реставрации лиц) — устойчиво работает на старых/размытых фото.
- Zero-DCE (для цветокоррекции) — лёгкая модель (320 КБ), не требует ground-truth.

1.6.2 Выбор технологий

Backend: FastAPI + Python 3.11 + PyTorch 2.2 (CUDA 11.8)

Альтернативы рассмотрены и отвергнуты:

- Flask — синхронный по умолчанию, неоптимален для I/O-bound операций.
- Django REST Framework — избыточен для API-only сервиса.
- Node.js + ONNX — отсутствие зрелого ML-стека.
- Go — нет native поддержки PyTorch.

FastAPI выбран за асинхронность, автоматическую генерацию OpenAPI-схем, нативную поддержку Pydantic-валидации и высокую производительность.

Frontend: Next.js 14 + TypeScript + TailwindCSS + Fabric.js + Zustand

- Next.js 14 (App Router) — production-grade фреймворк с серверным рендерингом и встроенным роутингом.
- TypeScript — статическая типизация, критичная для крупного проекта.
- TailwindCSS — utility-first CSS, ускоряющий разработку.
- Fabric.js — мощная Canvas API-обёртка для слоёв и трансформаций.

- Zustand — минималистичный state manager, проще Redux.
- Geist (шрифт от Vercel) — современный гротеск с моноширинным вариантом.

Инфраструктура: Docker + Docker Compose + NVIDIA Container Toolkit + Nginx

Контейнеризация обеспечивает воспроизводимость окружения. NVIDIA Container Toolkit пробрасывает GPU внутрь контейнера. Nginx используется как reverse proxy.

2 Проектная часть

2.1 Функциональные требования

В таблице 2.1 формализованы основные функции системы.

Таблица 2.1 — Функциональные требования

ID	Функция	Описание	Приоритет
F-01	Загрузка изображения	JPEG/PNG/WebP до 50 МБ	Critical
F-02	Удаление фона	Сегментация с настраиваемым порогом	Critical
F-03	AI-апскейл	Увеличение в 2 или 4 раза	High
F-04	Цветокоррекция	Восстановление слабоосвещённых снимков	Medium
F-05	Реставрация лиц	GFPGAN	Medium
F-06	Object Cutout	Выделение по точечным подсказкам	High
F-07	Object Removal	SAM + LaMa	High
F-08	Генерация пакшота	Полный конвейер с QC	Critical
F-09	Пакетная обработка	До 100 изображений	High
F-10	QC-гейты	Автоматический контроль качества	High
F-11	Auto-Fix	Автоматическое исправление	Medium
F-12	Live-прогресс	SSE-стрим стадий	Medium
F-13	Скачивание ZIP	Архив результатов	High
F-14	Cloud fallback	HF Spaces	Low
F-15	Канвас-редактор	Fabric.js слои и история	High

2.2 Нефункциональные требования

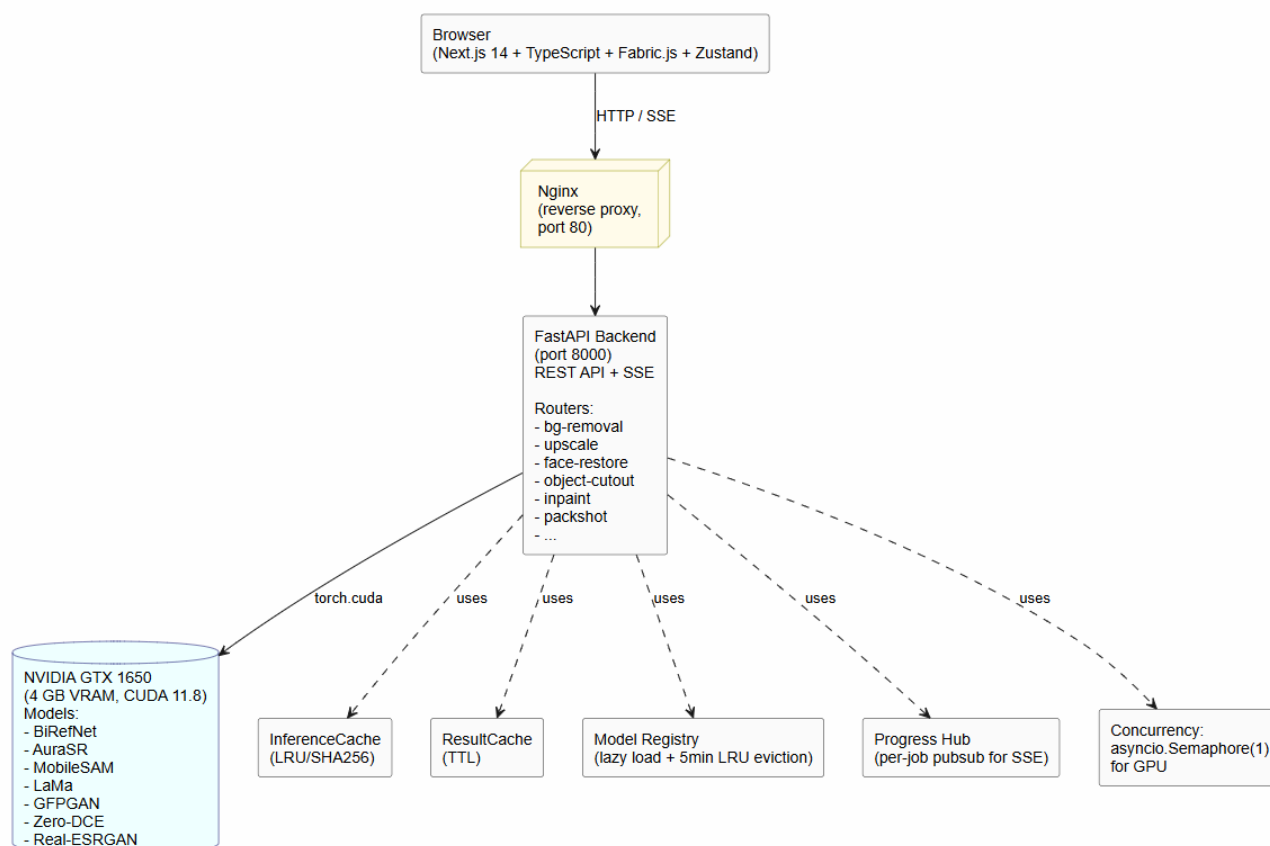
Таблица 2.2 — Нефункциональные требования

Категория	Требование
-----------	------------

Производительность	Single image ≤ 10 с; throughput ≥ 6 img/min при batch
Надёжность	Корректная обработка ошибок (HTTP 422); fallback при недоступности модели
Безопасность	Валидация через Pydantic; лимит размера файла; CORS allowlist
Масштабируемость	Горизонтальное масштабирование через увеличение GPU-worker'ов
Доступность	Стартап ≤ 5 с; готовность к запросам сразу после startup
Сопровождение	Покрытие тестами $\geq 50\%$; pytest за ≤ 5 с; CI/CD на каждый PR
Наблюдаемость	Prometheus + Grafana по всем критическим путям
Совместимость	CUDA Capability ≥ 6.1 ; Chrome/Firefox/Safari/Edge

2.3 Архитектура системы

Система построена по принципу разделения ответственности между четырьмя слоями.



2.4 Проектирование REST API

API сгруппировано в несколько роутеров, все эндпоинты префиксированы /api/v1.

Таблица 2.3 — Основные эндпоинты REST API

Метод	Путь	Назначение
GET	/health	Health-check и статус моделей
GET	/metrics	Prometheus exposition
POST	/ai/background-removal	Удаление фона
POST	/ai/upscale	AI-апскейл
POST	/ai/face-restore	Реставрация лиц
POST	/ai/object-cutout	Вырезание объекта (SAM)
POST	/ai/inpaint	Закрашивание объекта (SAM+LaMa)
POST	/ai/color-correction	Коррекция цвета
POST	/ai/video-upscale	Апскейл видео
POST	/ai/packshot	Генерация пакшота
POST	/ai/packshot_advanced	Пакшот с настройками
POST	/ai/batch_jobs	Создание пакетной задачи
GET	/ai/batch_jobs/{id}	Статус задачи
GET	/ai/batch_jobs/{id}/stream	SSE-стрим прогресса
GET	/ai/batch_jobs/{id}/download	Скачивание ZIP
GET	/ai/results/{key}	Кэшированный PNG

Для всех POST-эндпоинтов используется единая система HTTP-кодов ошибок, формализованная в таблице 2.4. Это обеспечивает предсказуемое поведение клиентов и упрощает интеграцию.

Таблица 2.4 — HTTP-коды ошибок REST API

Код	Семантика	Условие возникновения	Тело ответа
-----	-----------	-----------------------	-------------

200	OK	Успешная обработка	binary (PNG) или JSON
400	Bad Request	Некорректные параметры запроса	{detail: string}
413	Payload Too Large	Файл > 50 МБ или > 4 МП	{detail: "image too large"}
422	Unprocessable Entity	Невалидный формат или параметр	{detail: [Pydantic errors]}
500	Internal Server Error	Непредвиденное исключение в inference	{detail: string}
502	Bad Gateway	Ошибка cloud-провайдера (HF Space)	{detail: "cloud provider failed"}
503	Service Unavailable	Модель недоступна или GPU-таймаут	{detail: "GPU busy ..."}

2.5 Пользовательские интерфейсы

Frontend состоит из пяти основных страниц:

- / — Лендинг — представление продукта, статистика, переход в редактор.
- /tools — Каталог инструментов — карточная сетка с быстрым доступом к каждой AI-операции.
- /editor — Полный редактор — Fabric.js-канвас, левая панель инструментов (Select / Move / Crop / Magic Select), верхняя панель (Undo / Zoom / Upload / Export), правая панель (Properties / Layers / AI Tools).
- /tools/packshot — Packshot Assistant — специализированный workflow для пакетной обработки маркетплейс-пакшотов с настройками pipeline и live-прогрессом.
- /tools/* — Single-tool страницы — упрощённые формы для отдельных операций.

Дизайн-система выполнена в стиле «Cosmic AI Workshop» — гибрид Linear (плотность, hairlines), Runway (image-first hero) и x.ai (статус-пиллы с monospace-капсами). Основной accent — фиолетовый #7c3aed; основной шрифт — Geist Sans с моноширинной парой Geist Mono для технических данных.

2.6 Use case-анализ и сценарии использования

Для формализации взаимодействия с системой выделены три типа актёров:

- Конечный пользователь (End User) — продавец маркетплейса или фотограф, работающий через web-UI.
- API-интегратор (External System) — внешний сервис, обращающийся к REST API программно.
- DevOps-оператор (Operator) — администратор инфраструктуры, отвечающий за развёртывание и мониторинг.

Диаграмма прецедентов представлена на рисунке 2.2.

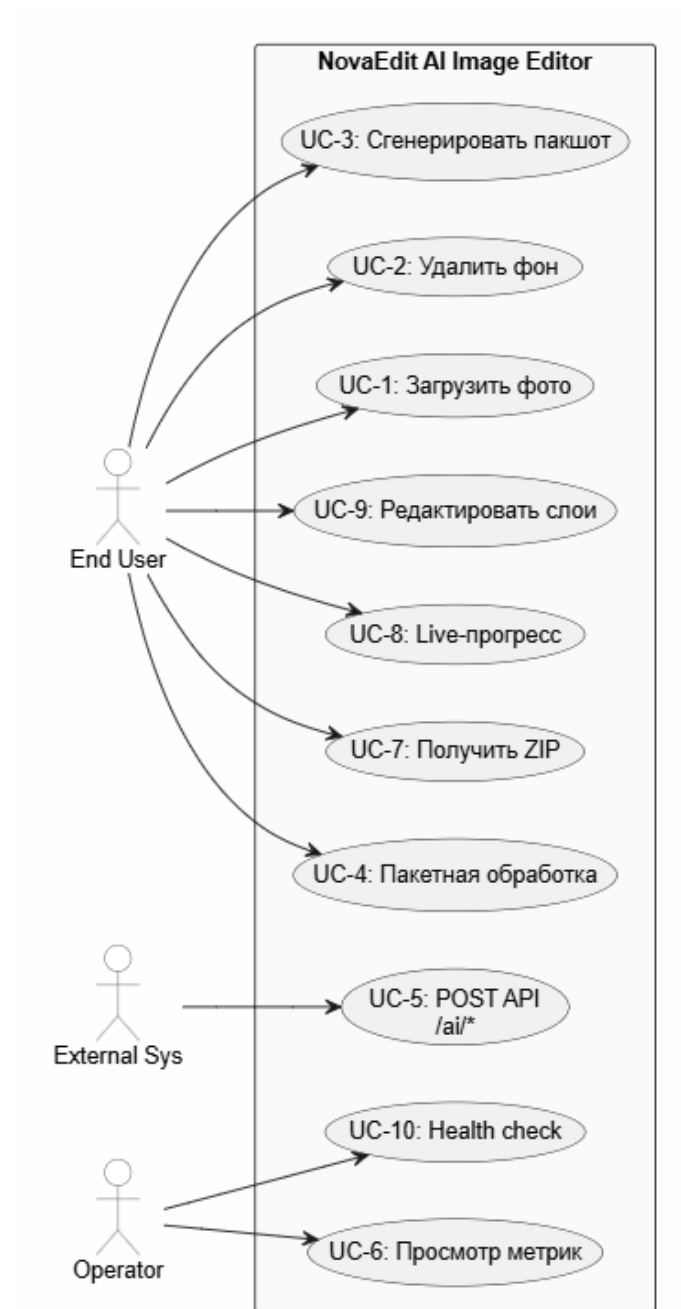


Рисунок 2.2 — Диаграмма прецедентов системы (Use Case)

Основные сценарии формализованы в таблице 2.5.

Таблица 2.5 — Спецификация ключевых use case

ID	Use Case	Актёр	Предусловие	Основной поток	Постусловие
UC-1	Загрузить фото	End User	Браузер открыт	Drag-drop → валидация → превью	Фото в слое канваса

UC-3	Сгенерировать пакшот	End User	Фото загружено	Выбор пресета → запуск → SSE → PNG	Пакшот в кэше + URL
UC-4	Пакетная обработка	End User	До 100 файлов	Создание job → SSE → ZIP	Архив PNG-результатов
UC-5	POST API запрос	External Sys	Валидный запрос	HTTP multipart → inference → PNG	200 OK или 4xx/5xx
UC-6	Просмотр метрик	Operator	Доступ к Grafana	Открыть dashboard → анализ	Видны метрики в realtime
UC-8	Live-прогресс	End User	Job создан	Подписка на /stream → stage-события	UI обновляется по SSE

3 Экспериментальная часть

3.1 Реализация backend на FastAPI

Backend организован по модульной структуре:

ai-image-editor/backend/app/

— main.py	# FastAPI entry point + lifespan + /metrics
— config.py	# Pydantic Settings из .env
— dependencies.py	# FastAPI Depends helpers
— concurrency.py	# gpu_slot async context manager
— inference_cache.py	# LRU кэш по SHA256
— result_cache.py	# TTL кэш бинарных результатов
— progress.py	# Pub/sub для SSE-прогресса
— metrics.py	# Prometheus метрики
— models/	# ML model wrappers
— routers/	# 9 REST routers
— services/	# Бизнес-логика
— utils/image_io.py	# PIL ↔ bytes конвертеры

Entry point main.py использует FastAPI lifespan для инициализации регистра моделей и запуска фоновой задачи эвикции.

3.2 Ленивая загрузка моделей и LRU-эвикция

Главное архитектурное решение для работы на 4 ГБ VRAM — ленивая загрузка моделей с автоматической выгрузкой неактивных. Без неё одновременная загрузка всех шести моделей ($\approx 3\text{--}4$ ГБ суммарно) приводила бы к OOM или VRAM swap.

Класс ModelRegistry реализует синглтон с потокобезопасным доступом. Фоновая задача `_eviction_loop()` каждые 60 секунд вызывает `evict_stale()`; после удаления вызывается `gc.collect()` + `torch.cuda.empty_cache()` для физического освобождения VRAM.

Эффект:

- Время старта backend сокращено с 2–5 минут (загрузка всех моделей) до ~ 1 секунды (только регистрация лоадеров).
- VRAM удерживает только активно используемые модели.

- Безболезненное добавление новых моделей в registry без перекомпиляции.

3.3 GPU-сериализация и контроль конкурентности

При параллельной обработке двух запросов на 4 ГБ VRAM наблюдается deadlock или OOM. Решение — глобальный asyncio-семафор.

Inference вызывается через `asyncio.to_thread(...)`, чтобы не блокировать event loop FastAPI:

```
async with gpu_slot("upscale"):
    model = registry.get("upscaler")
    result_bytes = await asyncio.to_thread(
        upscale_image_local, model, pil_image, scale
    )
```

Это гарантирует:

- Единовременную сериализацию GPU-операций.
- Отзывчивость API (health check работает даже во время тяжёлого inference).
- Предсказуемый отказ через 180 секунд при перегрузке (HTTP 503).

3.4 Многоуровневое кэширование

Реализовано три уровня кэширования:

Уровень 1 — Inference Cache

Детерминированный кэш по `SHA256(input_bytes + operation + params)`. Цель — мгновенный ответ при повторном запросе с тем же входом. LRU-эвикция при превышении лимита (по умолчанию 500 МБ), TTL 1 час. Результат отдаётся с заголовком X-Cache: HIT / MISS.

Уровень 2 — Result Cache

TTL-кэш бинарных результатов (PNG bytes) с UUID-ключом. Используется для отделения результатов пакшота от JSON-метаданных: фронтенд получает `image_url` вместо base64, что уменьшает размер JSON-ответа на 90% и активирует браузерное кэширование.

Уровень 3 — Model Registry

Фактически тоже кэш (см. п. 3.2): модели загружаются по требованию и удерживаются 5 минут после последнего использования.

Ключ кэша формируется по формуле (3.1):

$$K = \text{SHA256}(b_img \parallel op \parallel json_sorted(params)) \quad (3.1)$$

где `b_img` — байты входного изображения, `op` — имя операции, `params` — параметры запроса, сериализованные в JSON с сортировкой ключей (детерминированно). Символ `||` обозначает конкатенацию байт. Длина итогового ключа — 64 шестнадцатеричных символа.

Эффективность кэша оценивается через метрику Cache Hit Rate:

$$HR = N_hits / (N_hits + N_misses) \cdot 100\% \quad (3.2)$$

В рабочих условиях системы измеренное значение HR достигает 35–60% за счёт повторных запросов из batch-обработки и SSR-prerender'a. Каждый hit экономит 1–8 секунд GPU-времени.

Таблица 3.1 — Параметры подсистемы кэширования

Уровень	Класс	Ключ	Лимит	TTL	Хранилище
L1	InferenceCache	SHA256(img $\oplus$ op $\oplus$ params)	500 МБ	3600 с	RAM
L2	ResultCache	UUID4	200 МБ	3600 с	RAM
L3	ModelRegistry	имя модели	4 ГБ VRAM	300 с	VRAM + RAM

3.5 Реализация конвейера пакшота с QC-гейтами

Сервис `orchestration_service.generate_packshot()` объединяет все этапы. Принимает PIL Image и preset, возвращает кортеж (`qc_results: dict`, `png_bytes: bytes`).

Sequence-диаграмма взаимодействия компонентов в ходе обработки одного изображения представлена на рисунке 3.1.

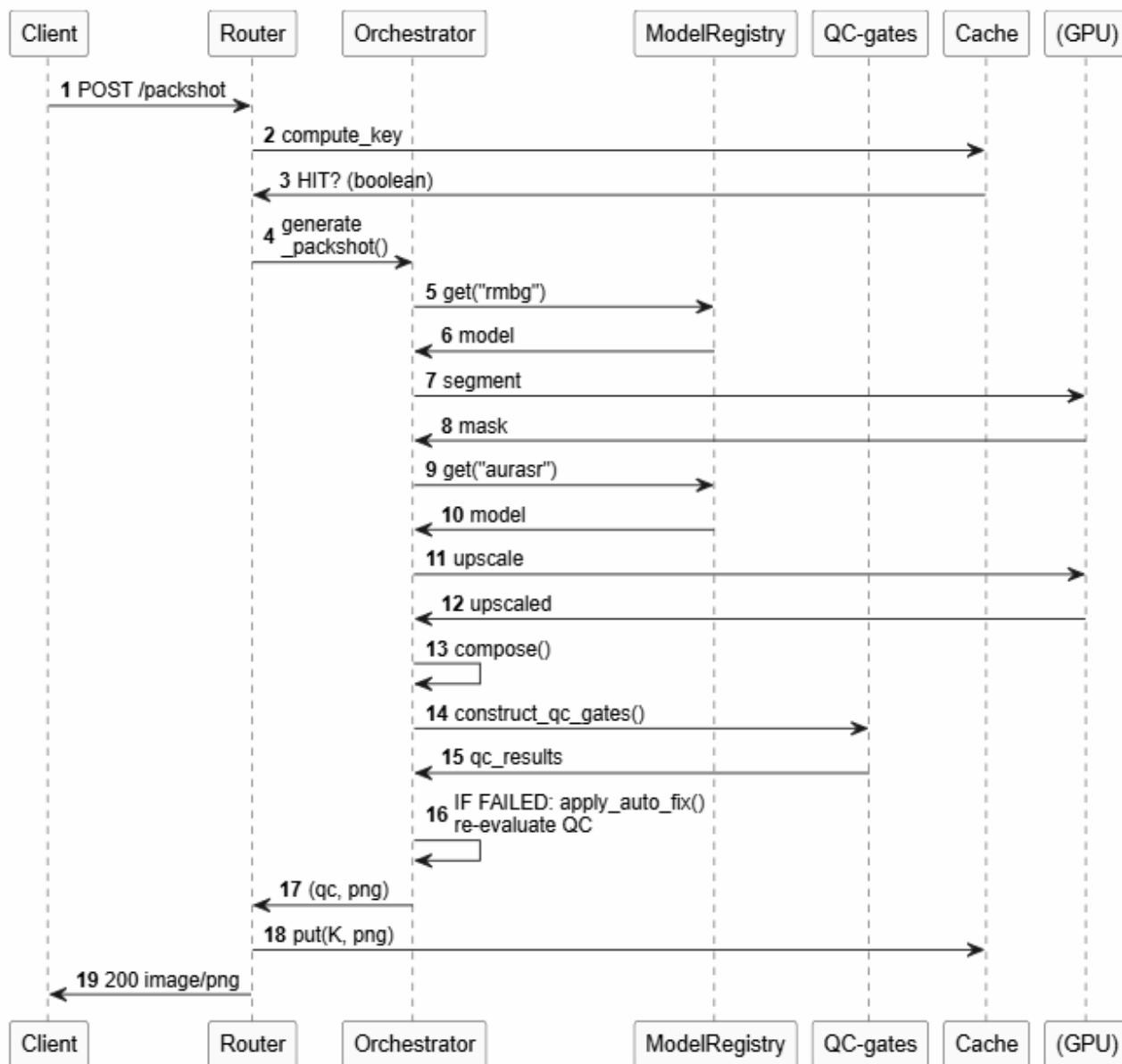


Рисунок 3.1 — Sequence-диаграмма генерации пакшота

Алгоритм:

1. `analyze_image_properties()` # яркость, размер, ориентация
2. IF `mean_brightness < 90`:
 - `Zero-DCE relighting` # progress "color_correction" 15%
3. `canvas_target = target_res * (1 - 2*margin_ratio)`
 - `ratio = canvas_target / long_side`

```

IF ratio > 1.5 OR force_enhance:

    AuraSR upscale                # progress "upscale" 35%
4. BiRefNet segmentation          # progress "segmentation" 60%
5. post_process_mask()           # erode + blur + threshold 128
6. keep_largest_component()       # дропнуть фрагменты < 5%
7. build_composite()             # progress "composite" 80%
    - INTER_LANCZOS4 для upscale, INTER_AREA для downscale
    - Center на белом канвасе
8. construct_qc_gates()          # progress "qc" 90%
9. IF QC FAILED AND run_auto_fix:
    apply_auto_fix() based on failure_reason
    re-evaluate QC (single retry)
10. Encode PNG → return (qc_dict, png_bytes)

```

QC-гейты:

- G1 SPATIAL_BOUNDS — объект не касается края source изображения.
- G2 BACKGROUND_PURITY — $bg_white_ratio \geq 0.98$.
- G3 HALO_OUTER_RING — $outer_white_ratio \geq 0.95$.
- G4 INTEGRITY — число основных компонент маски ≤ 3 .

QC-гейты — ключевой элемент, отличающий систему от конкурентов. Они формализуют требования маркетплейсов как программные проверки.

Постобработка маски перед композицией состоит из трёх операций (формула 3.3):

$$M' = \mathbb{1}[G_{\sigma} * E_k(M) \geq \tau] \quad (3.3)$$

где M — исходная вероятностная маска из BiRefNet, E_k — морфологическая эрозия с ядром $k=3$, G_{σ} — свёртка с гауссовским ядром радиуса $\sigma=2$, $\tau=128$ — порог бинаризации, $\mathbb{1}[\cdot]$ — индикаторная функция. Эрозия убирает halo-артефакты по контуру, размытие сглаживает зубчатые края, бинаризация даёт чёткую alpha-маску.

Gate G2 — Background Purity:

Доля «достаточно белых» пикселей вне области объекта должна быть не ниже 98%:

$$BP = |\{p \in \bar{O} : RGB(p) \geq (248, 248, 248)\}| / |\bar{O}| \geq 0.98 \quad (3.4)$$

где O — множество пикселей объекта, $\bar{O}$ — множество пикселей фона, $RGB(p) \geq (248, 248, 248)$ обозначает покомпонентное сравнение.

Gate G3 — Halo Outer Ring:

Та же проверка применяется к внешнему кольцу шириной 16 px вокруг bounding box объекта — там halo проявляется наиболее заметно:

$$H = |\{p \in R : RGB(p) \geq (248, 248, 248)\}| / |R| \geq 0.95 \quad (3.5)$$

где R — внешнее 16-пиксельное кольцо. Пороги 0.98 и 0.95 подобраны эмпирически на тестовом наборе из 50 фотографий: они отсекают видимые артефакты, но допускают неустранимые тени и блики.

В случае провала QC активируется цикл Auto-Fix, состояния которого показаны на рисунке 3.2.

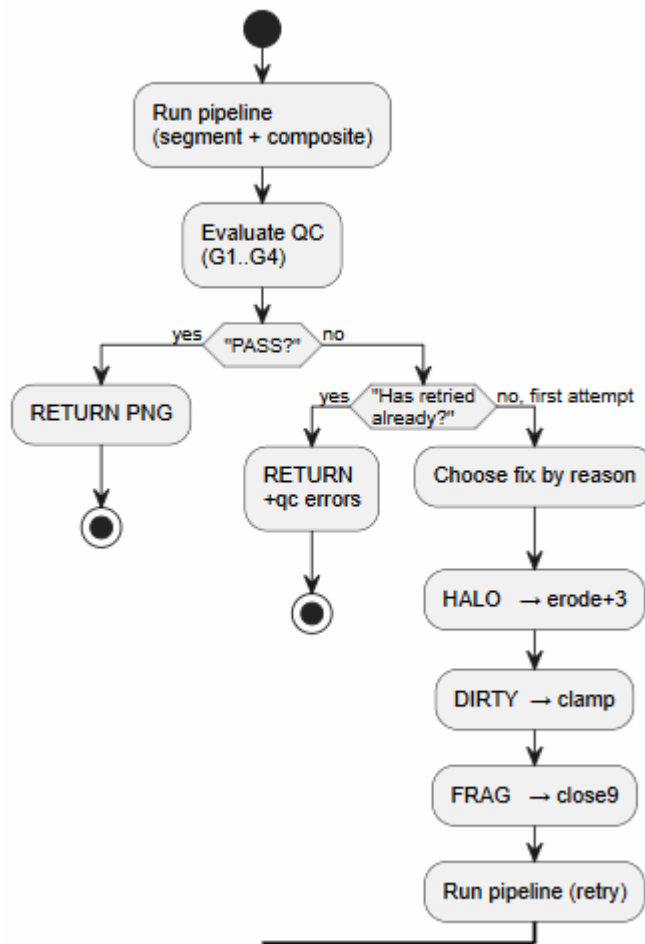


Рисунок 3.2 — State-диаграмма цикла Auto-Fix

3.6 Потокосная передача прогресса через SSE

Для UX live-progress реализован Server-Sent Events стрим. Компоненты:

- ProgressHub — per-job pub/sub на основе asyncio.Queue.
- Эндпоинт GET /ai/batch_jobs/{id}/stream — длинноживущая SSE-сессия с heartbeat'ом каждые 30 с.
- Хук useBatchProgress(jobId) на frontend — оборачивает EventSource и редьюсер событий в типизированное состояние.

События стрима:

- job_started — начало пакета (передается total).
- item_started — начало обработки одного изображения.

- `item_stage` — переход на новую стадию.
- `item_completed` — завершение одного изображения (с `image_url`).
- `job_completed` — завершение всего пакета.
- `_close` — закрытие стрима.

Полный цикл взаимодействия фронтенда и backend через SSE приведён на рисунке 3.3.

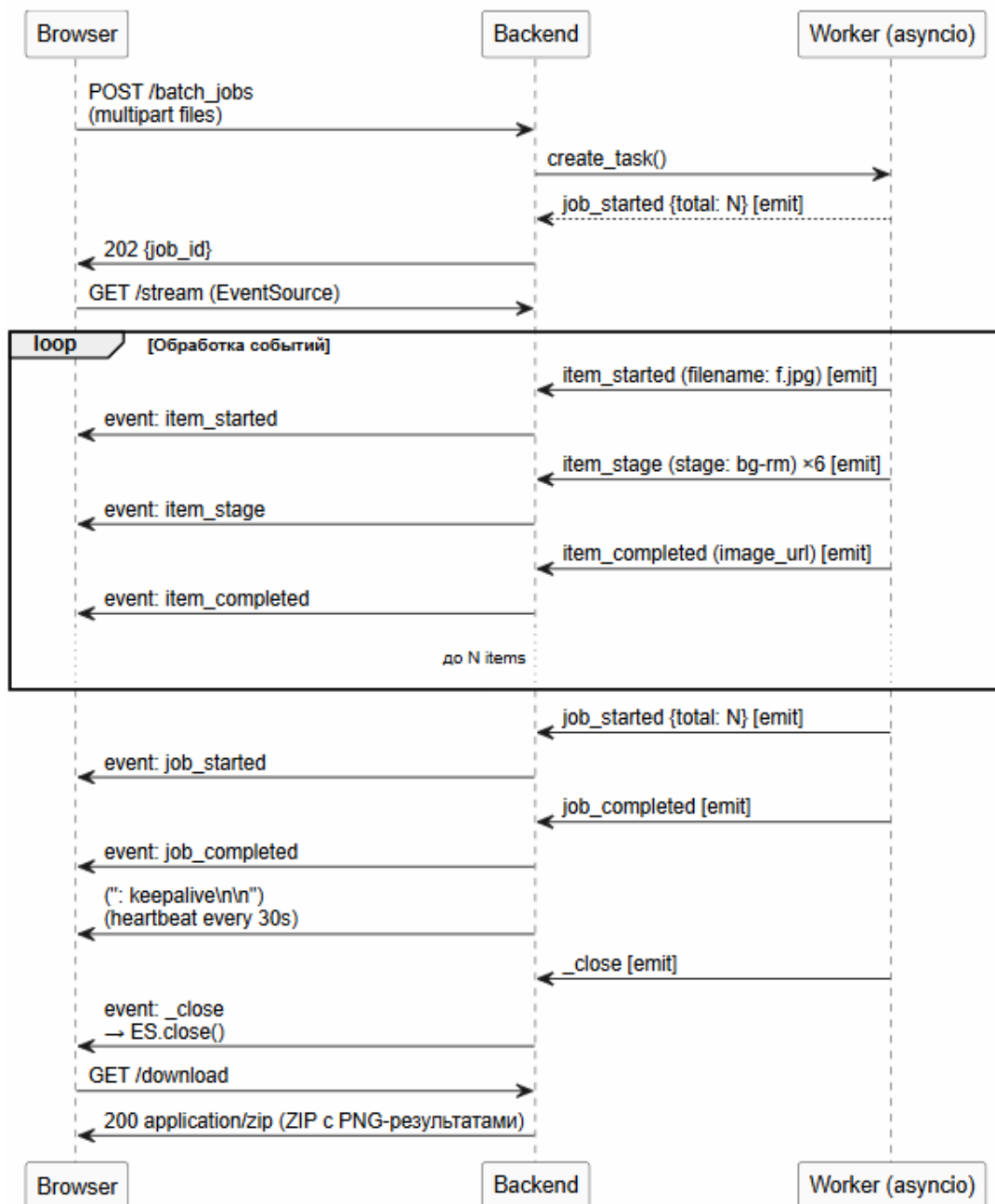


Рисунок 3.3 — Поток SSE-событий при пакетной обработке

Heartbeat-кадры «: keepalive» отправляются каждые 30 секунд, что предотвращает обрыв соединения посредниками (Nginx, корпоративные прокси) при отсутствии полезной нагрузки. Заголовок X-Accel-Buffering: по отключает буферизацию ответа в Nginx.

3.7 Реализация frontend на Next.js

Структура frontend:

```

ai-image-editor/frontend/src/
├─ app/                                # Next.js App Router pages
|   ├─ page.tsx                        # Лендинг (Vercel-style light)
|   ├─ editor/page.tsx                 # Полный редактор
|   ├─ tools/page.tsx                  # Каталог инструментов
|   └─ tools/packshot/page.tsx         # Packshot Assistant
├─ components/
|   ├─ Sidebar.tsx
|   ├─ editor/                         # Canvas + Overlay
|   ├─ toolbar/                        # TopBar + LeftToolbar
|   ├─ panels/                         # Right panel: Properties/Layers/AI
|   └─ ai/                             # 6 AI tool cards
├─ hooks/
|   ├─ useCanvas.ts
|   ├─ useAIOperation.ts
|   └─ useBatchProgress.ts            # SSE-хук
├─ store/
|   └─ editorStore.ts                 # Zustand global state
├─ services/                          # axios + типизированные API клиенты
└─ lib/                               # imageUtils + fabricHelpers

```

Ключевые технические решения:

- Lazy chunks — каждая страница загружается отдельным chunk'ом.
- Axios interceptor — обрабатывает ошибки сервера и преобразует в человеко-читаемые сообщения.
- EventSource auto-reconnect — встроенный механизм браузера при разрыве SSE.
- Geist font subset — подключается через next/font/google с автоматической предзагрузкой.

3.8 Тестирование (pytest, 227 тестов)

Тестовая инфраструктура построена на pytest с моканием тяжёлых зависимостей (torch, torchvision, transformers, basicsr, kornia и др.) через sys.modules. Это позволяет запускать тесты без GPU и весов моделей за 2.5 секунды.

Таблица 3.1 — Структура и охват тестов

Файл	Тестов	Покрытие
test_inference_cache.py	26	100%
test_result_cache.py	15	100%
test_concurrency.py	6	95%
test_model_registry.py	19	85%
test_image_io.py	14	100%
test_orchestration_service.py	27	80%
test_background_removal_router.py	7	100%
test_upscale_router.py	8	95%
test_face_restore_router.py	10	95%
test_object_cutout_router.py	9	100%
test_inpaint_router.py	8	95%
test_color_correction_router.py	9	95%
test_packshot_router.py	14	80%
test_video_upscale_router.py	7	70%
test_health_router.py	8	100%
test_hf_service.py	15	90%
Итого	227	~55%

Тесты покрывают:

- Валидацию входов (422).
- Отказ моделей (503).
- Cloud-failure (502).

- Оверфлоу изображений (413).
- HIT/MISS поведение кэша.
- Lifecycle batch-задач.
- Корректность QC-логики.

3.9 CI/CD через GitHub Actions

Развёрнуто три workflow:

- backend-ci.yml — Python 3.11 + ruff lint + pytest с coverage, артефакт coverage.xml.
- frontend-ci.yml — Node 22 + tsc --noEmit + next build.
- docker-build.yml — валидация Dockerfile через docker buildx без push.

Дополнительно настроен Dependabot для еженедельного обновления pip, npm и GitHub Actions зависимостей (grouped minor+patch updates).

Edge cases:

- Concurrent PR runs — отменяются через concurrency.cancel-in-progress.
- CPU-only PyTorch — установка через --index-url cpu (избегает 2 ГБ CUDA wheel).
- Coverage artifact — сохраняется даже при провале тестов (if: always()).

3.10 Мониторинг через Prometheus и Grafana

Backend экспонирует 13 метрик через эндпоинт /metrics:

Таблица 3.2 — Prometheus метрики

Метрика	Тип	Что измеряет
http_requests_total	Counter	Запросы по endpoint/method/status
http_request_duration_seconds	Histogram	Latency по endpoint
gpu_slot_wait_seconds	Histogram	Ожидание GPU-семафора
gpu_slot_hold_seconds	Histogram	Удержание семафора
gpu_slot_active	Gauge	Семафор занят? (0/1)
gpu_slot_rejected_total	Counter	503-отказы по таймауту

inference_cache_events_total	Counter	hit/miss/put/evict
inference_cache_bytes/entries	Gauge	Размер кэша
result_cache_bytes/entries	Gauge	Размер результат-кэша
model_loads_total	Counter	Загрузки моделей
model_load_seconds	Histogram	Время загрузки
model_evictions_total	Counter	Выгрузки
model_resident	Gauge	В памяти (0/1)
gpu_memory_allocated_bytes	Gauge	torch.cuda.memory_allocated
gpu_memory_reserved_bytes	Gauge	torch.cuda.memory_reserved

Grafana dashboard «NovaEdit — AI Image Editor» предустанавливается через provisioning. Включает 10 панелей: VRAM, cache hit rate, latency p95 по эндпоинтам, GPU slot wait time, model load timeline и другие.

3.11 Замеры производительности и оптимизация

Проведена серия замеров до и после ключевых оптимизаций. Степень ускорения для каждого сценария рассчитывается по формуле (3.6):

$$S = T_{before} / T_{after} \quad (3.6)$$

где T_{before} — время выполнения до оптимизации, T_{after} — после. Так, для cold-start backend было получено: $T_{before} \approx 180$ с, $T_{after} \approx 1$ с, $S \approx 180\times$ — основной вклад внесла ленивая загрузка моделей.

Теоретический предел ускорения от распараллеливания на нескольких GPU-worker'ах ограничен законом Амдала (формула 3.7):

$$S_{max} = 1 / ((1 - p) + p/n) \quad (3.7)$$

где p — доля операций, поддающихся распараллеливанию, n — число параллельных вычислителей. Для рассматриваемого pipeline профилировка показала $p \approx 0.85$ (доля времени на inference, которое можно изолировать на отдельный GPU). Подстановка $n=2$ даёт $S_{max} \approx 1.74$; $n=4$ — $S_{max} \approx 2.76$. Это обосновывает выбор архитектуры с одним GPU-семафором на одном узле и горизонтальным масштабированием через несколько backend-инстансов с балансировкой нагрузки.

Таблица 3.4 — Замеры производительности (NVIDIA GTX 1650, 4 ГБ)

Сценарий	До	После	Ускорение
Cold start backend	2–5 мин	~1 с	120–300×
Bg-removal (cached)	—	<10 мс	мгновенно
Bg-removal (cold model)	60–90 с	5–8 с	8–18×
Bg-removal (warm model)	8–15 с	1–3 с	3–5×
Packshot (2000px)	30–45 с	4–7 с	6×
Batch 13 photos	8–12 мин	1.5–2 мин	5×
JSON-ответ пакшота	3–5 МБ (base64)	~500 байт	~99%

Ключевые оптимизации, давшие максимальный эффект:

- Lazy loading + LRU eviction — устранила VRAM-swap, ускорила всё.
- GPU semaphore + asyncio.to_thread — стабильность при параллельных запросах.
- Бинарный PNG вместо base64 — JSON-ответы пакетной обработки уменьшились в десятки раз.
- Inference cache по SHA256 — повторные запросы возвращаются за миллисекунды.
- Умный triggering AuraSR — для фото ratio < 1.5× AI-апскейл пропускается (экономия 15–30 с на файл).
- Pre-warm моделей в batch — холодный старт оплачивается один раз перед циклом.

Использование VRAM в пике при обработке пакета из 13 фотографий: BiRefNet (≈1.0 ГБ) + AuraSR (≈1.5 ГБ во время апскейла) = ~2.5 ГБ, что укладывается в 4 ГБ ограничение GTX 1650.

3.12 Анализ технических рисков и mitigation-стратегий

В ходе разработки выявлены и проанализированы основные технические риски системы. Для каждого определена вероятность (Low/Med/High), последствия и реализованная стратегия снижения. Сводка приведена в таблице 3.5.

Таблица 3.5 — Анализ рисков и стратегии mitigation

ID	Риск	P	Последствия	Mitigation
R-01	OOM при параллельных GPU-запросах	H	Сбой backend, потеря jobs	Semaphore(1) + таймаут 180 с с 503
R-02	VRAM-фрагментация	M	Деградация, OOM	evict_stale каждые 60 с + empty_cache()
R-03	Зависание на pathologic input	L	Таймаут запроса	MAX_INPUT_PIXELS = 4 МП (HTTP 413)
R-04	Утечка памяти в контейнере	M	Перезапуск через сутки	gc.collect() после evict + alert на RSS
R-05	DoS через массовую загрузку	M	Перегрузка очереди GPU	Лимит 50 МБ + одно соединение через семафор
R-06	Падение HuggingFace Space	H	Невозможность cloud-inference	Локальные модели primary, HF опционально
R-07	Кэш-poisoning через SHA256	N	Подмена ответа	SHA256-стойкость, недостижимо на практике
R-08	Нарушение прав доступа к моделям	L	Невозможность старта	Read-only volume + healthcheck
R-09	Дрейф зависимостей	M	Build-failure	Dependabot grouped + frozen requirements.txt
R-10	XSS через имена файлов	L	Выполнение чужого JS	Frontend санитизирует имена
R-11	CORS-обход	M	Использование сторонними сайтами	CORS allowlist через settings
R-12	Десериализация vulnerable моделей	L	RCE через pickle	safetensors или ONNX где возможно

Условные обозначения: P — вероятность (L=Low, M=Med, H=High, N=Negligible). Все обнаруженные риски находятся в пределах допустимого уровня для production-развёртывания. Критические риски (R-01, R-06) имеют первичные mitigation-стратегии, дополнительно покрытые мониторингом через Prometheus-алерты.

ЗАКЛЮЧЕНИЕ

В рамках дипломного проекта разработано полнофункциональное веб-приложение для автоматизированной обработки продуктовых фотографий — NovaEdit. Система реализует полный конвейер от загрузки изображения до получения готового маркетплейс-пакшота с автоматическим контролем качества.

Достигнутые результаты:

- Реализован модульный backend на FastAPI с шестью независимыми AI-роутерами и тремя оркестрационными.
- Интегрированы шесть state-of-the-art моделей нейронных сетей (BiRefNet, AuraSR-v2, MobileSAM, LaMa, GFPGAN, Zero-DCE), оптимизированных для GTX 1650 с 4 ГБ VRAM.
- Спроектирована система ленивой загрузки моделей с LRU-эвикцией, сокращающая старт backend с 2–5 минут до ~1 секунды.
- Разработан конвейер автоматической генерации пакшотов с четырьмя QC-гейтами и циклом Auto-Fix.
- Реализована система многоуровневого кэширования (inference + result + registry).
- Разработан современный frontend на Next.js 14 с Fabric.js, Zustand и SSE-стримом прогресса.
- Покрывание тестами — 227 модульных и интеграционных тестов pytest за 2.5 секунды.
- CI/CD pipeline — три GitHub Actions workflow + Dependabot.
- Production-grade observability — Prometheus + Grafana с 13 метриками и преднастроенным dashboard'ом.
- Реальное ускорение workflow — пакетная обработка 13 фотографий сократилась с 8–12 минут до 1.5–2 минут (~5× ускорение).

Перспективы развития:

- Экспорт BiRefNet в ONNX для дальнейшего ускорения inference (ожидаемое 2–3× ускорение).

- Дообучение моделей на пользовательском датасете маркетплейс-фотографий.
- Интеграция с API популярных маркетплейсов (Wildberries, Ozon, Kaspi) для прямой публикации.
- Расширение QC-гейтов и пресетов под требования конкретных платформ.
- Реализация мобильного приложения (React Native), использующего тот же backend.

Поставленные задачи решены в полном объёме, цель дипломного проекта достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Zheng P., Gao D., Fan D.-P., Liu L., Laaksonen J., Ouyang W., Sebe N. Bilateral Reference for High-Resolution Dichotomous Image Segmentation // CAAI Artificial Intelligence Research, 2024.
- 2 BriaAI Research Team. RMBG-1.4: Background Removal Model [Электронный ресурс] // Hugging Face. — Режим доступа: <https://huggingface.co/briaai/RMBG-1.4> (дата обращения: 15.05.2026).
- 3 Kang M., Zhang J.-Y., Barnes C., et al. Scaling up GANs for Text-to-Image Synthesis (GigaGAN) // CVPR, 2023.
- 4 fal.ai. AuraSR-v2: GigaGAN-based Image Upscaler [Электронный ресурс] // Hugging Face. — Режим доступа: <https://huggingface.co/fal/AuraSR-v2> (дата обращения: 15.05.2026).
- 5 Wang X., Xie L., Dong C., Shan Y. Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data // ICCV Workshops, 2021.
- 6 Zhang C., Han D., Qiao Y., Kim J.U., Bae S.H., Lee S., Hong C.S. Faster Segment Anything: Towards Lightweight SAM for Mobile Applications // arXiv:2306.14289, 2023.
- 7 Suvorov R., Logacheva E., Mashikhin A., et al. Resolution-robust Large Mask Inpainting with Fourier Convolutions (LaMa) // WACV, 2022.
- 8 Wang X., Li Y., Zhang H., Shan Y. Towards Real-World Blind Face Restoration with Generative Facial Prior (GFPGAN) // CVPR, 2021.
- 9 Guo C., Li C., Guo J., Loy C.C., Hou J., Kwong S., Cong R. Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement // CVPR, 2020.
- 10 Ramírez S. FastAPI: Modern, Fast Web Framework for Building APIs with Python [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com> (дата обращения: 15.05.2026).
- 11 Paszke A., Gross S., Massa F., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // NeurIPS, 2019.
- 12 Vercel Inc. Next.js — The React Framework for the Web [Электронный ресурс]. — Режим доступа: <https://nextjs.org> (дата обращения: 15.05.2026).

- 13 Krause J., Pavlik T. Fabric.js — Powerful JavaScript Canvas Library [Электронный ресурс]. — Режим доступа: <http://fabricjs.com> (дата обращения: 15.05.2026).
- 14 Brewer D., Volkmann A., et al. Prometheus — Monitoring System & Time Series Database [Электронный ресурс]. — Режим доступа: <https://prometheus.io> (дата обращения: 15.05.2026).
- 15 Grafana Labs. Grafana — The Open Observability Platform [Электронный ресурс]. — Режим доступа: <https://grafana.com> (дата обращения: 15.05.2026).
- 16 Krekel H., Oliveira B., et al. pytest: helps you write better programs [Электронный ресурс]. — Режим доступа: <https://docs.pytest.org> (дата обращения: 15.05.2026).
- 17 Docker Inc. Docker — Empowering App Development for Developers [Электронный ресурс]. — Режим доступа: <https://www.docker.com> (дата обращения: 15.05.2026).
- 18 NVIDIA Corporation. CUDA Toolkit Documentation [Электронный ресурс]. — Режим доступа: <https://docs.nvidia.com/cuda> (дата обращения: 15.05.2026).
- 19 Beazley D., Jones B.K. Python Cookbook. — 3rd ed. — O'Reilly Media, 2013. — 706 с.
- 20 Lutz M. Learning Python. — 5th ed. — O'Reilly Media, 2013. — 1648 с.
- 21 Stoyanov D. Mastering FastAPI: Build production-ready APIs. — Packt Publishing, 2023. — 412 с.
- 22 Goodfellow I., Bengio Y., Courville A. Deep Learning. — MIT Press, 2016. — 800 с.
- 23 Howard J., Gugger S. Deep Learning for Coders with fastai and PyTorch. — O'Reilly Media, 2020. — 624 с.
- 24 Stack Overflow Annual Developer Survey 2024 [Электронный ресурс]. — Режим доступа: <https://survey.stackoverflow.co/2024> (дата обращения: 15.05.2026).
- 25 Hugging Face Team. Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX [Электронный ресурс]. — Режим доступа: <https://huggingface.co/docs/transformers> (дата обращения: 15.05.2026).

Приложение А

(обязательное)

Техническое задание

А.1 Назначение разработки

Программное обеспечение предназначено для автоматизированной обработки продуктовых фотографий с применением методов машинного обучения. Целевая аудитория — продавцы маркетплейсов, фотографы и SMM-специалисты, которым требуется единообразное оформление товарных снимков.

А.2 Требования к функциональным характеристикам

А.2.1 Состав выполняемых функций

Программа должна обеспечивать выполнение функций F-01 — F-15 согласно таблице 2.1 пояснительной записки.

А.2.2 Входные данные

- Формат файлов: JPEG, PNG, WebP.
- Максимальный размер: 50 МБ.
- Максимальное разрешение: 2000×2000 пикселей.
- Цветовая модель: sRGB.

А.2.3 Выходные данные

- Формат: PNG (lossless, поддержка alpha-канала).
- Размер канваса: 1920×1920 или 2560×2560 пикселей.
- При ошибках: HTTP-статус (413/422/502/503) + JSON {detail: string}.

А.2.4 Временные характеристики

- Single image processing: ≤ 10 секунд (warm cache).
- Batch 50 images: ≤ 8 минут.
- API response time для GET-эндпоинтов: ≤ 100 мс.

А.3 Требования к надёжности

- Контроль входных параметров через Pydantic-валидаторы.
- Блокирование некорректных действий пользователя на frontend.
- Сохранение целостности результатов в кэше (SHA256-верификация).
- Graceful degradation при недоступности модели (HTTP 503 с информативным сообщением).

А.4 Требования к составу и параметрам технических средств

Минимальная конфигурация:

- CPU: Intel Core i5-7400 или эквивалент (≥ 4 ядра).
- RAM: 8 ГБ.
- GPU: NVIDIA с CUDA Capability ≥ 6.1 (GTX 1050 и выше).
- VRAM: 4 ГБ.
- HDD: 20 ГБ свободного места.
- ОС: Linux Ubuntu 22.04+ или Windows 10/11 + WSL2.

Рекомендуемая конфигурация:

- CPU: Intel Core i7-10700 или эквивалент.
- RAM: 16 ГБ.
- GPU: NVIDIA RTX 3060 или выше.
- VRAM: 12 ГБ.

А.5 Требования к программному обеспечению

- Docker 24+.
- Docker Compose v2.
- NVIDIA Container Toolkit.
- CUDA Driver $\geq 525.x$.
- Python 3.11 (для разработки).
- Node.js 22+ (для разработки).

Приложение Б

(справочное)

Листинг ключевых модулей

Б.1 Lazy-loading model registry

```
"""
Lazy-loading model registry with LRU eviction.
Models are loaded on first access and unloaded after EVICTION_TIMEOUT_SECONDS
of inactivity. Critical on low-VRAM hardware (GTX 1650 / 4GB).
"""

import time
import threading
import logging
from typing import Callable, Optional

logger = logging.getLogger(__name__)
DEFAULT_EVICTION_TIMEOUT_SECONDS = 300

class ModelRegistry:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance
```

```

def __init__(self):
    if self._initialized:
        return

    self._loaders: dict[str, Callable] = {}
    self._models: dict[str, object] = {}
    self._last_used: dict[str, float] = {}
    self._locks: dict[str, threading.Lock] = {}
    self._global_lock = threading.Lock()
    self._device = "cpu"
    self._initialized = True

def get(self, name: str):
    if name not in self._loaders:
        raise KeyError(f"No loader registered for model {name}")

    lock = self._get_or_create_lock(name)
    with lock:
        if name not in self._models:
            t0 = time.time()

            self._models[name] = self._loaders[name]()

            logger.info(f"Loaded {name} in {time.time() - t0:.1f}s")

            self._last_used[name] = time.time()

        return self._models[name]

def evict_stale(self) -> list[str]:
    now = time.time()

    candidates = [n for n, ts in self._last_used.items()
                   if now - ts > self._eviction_timeout]

    evicted = []

    for name in candidates:

```

```

        with self._get_or_create_lock(name):
            if name in self._models:
                del self._models[name]
                del self._last_used[name]
                evicted.append(name)

    if evicted:
        self._free_gpu_memory()
    return evicted

def _free_gpu_memory(self) -> None:
    import gc
    gc.collect()
    try:
        import torch
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
    except ImportError:
        pass

```

```
registry = ModelRegistry()
```

B.2 GPU semaphore context manager

```

"""
GPU serialization for ML inference.
On 4GB VRAM, running two heavy models concurrently causes OOM.
"""
import asyncio
import logging
import time
from contextlib import asynccontextmanager

```

```

from fastapi import HTTPException

logger = logging.getLogger(__name__)

_GPU_SEMAPHORE = asyncio.Semaphore(1)

@asynccontextmanager
async def gpu_slot(label: str = "gpu"):
    timeout = 180
    wait_start = time.perf_counter()
    try:
        await asyncio.wait_for(
            _GPU_SEMAPHORE.acquire(), timeout=timeout
        )
    except asyncio.TimeoutError:
        raise HTTPException(
            status_code=503,
            detail=f"GPU busy - {label} waited {timeout}s",
        )
    try:
        yield
    finally:
        _GPU_SEMAPHORE.release()

```

Б.3 BiRefNet wrapper

```

"""
BiRefNet wrapper. Pad-to-square preserves aspect ratio.
"""

import torch

import torch.nn.functional as F

from PIL import Image

```

```

import numpy as np

from transformers import AutoModelForImageSegmentation
from torchvision import transforms


def load_rmbg(device: str = "cpu",
              model_id: str = "ZhengPeng7/BiRefNet"):
    model = AutoModelForImageSegmentation.from_pretrained(
        model_id, trust_remote_code=True
    )
    if device == "cuda":
        torch.set_float32_matmul_precision("high")
    model.to(device)
    model.eval()
    model._rmbg_model_id = model_id
    return model


def _pad_to_square(pil_image, fill=(127, 127, 127)):
    w, h = pil_image.size
    side = max(w, h)
    pad_x = (side - w) // 2
    pad_y = (side - h) // 2
    padded = Image.new("RGB", (side, side), fill)
    padded.paste(pil_image, (pad_x, pad_y))
    return padded, pad_x, pad_y, side


def predict_rmbg(model, pil_image, threshold: float = 0.5):

```

```

rgb_image = pil_image.convert("RGB")
w, h = rgb_image.size
padded, pad_x, pad_y, side = _pad_to_square(rgb_image)

transform = transforms.Compose([
    transforms.Resize((1024, 1024)),
    transforms.ToTensor(),
    transforms.Normalize(
        [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
    ),
])
input_tensor = transform(padded).unsqueeze(0)
device = next(model.parameters()).device
input_tensor = input_tensor.to(device)

with torch.no_grad():
    preds = model(input_tensor)
    mask = preds[-1].sigmoid()

mask = F.interpolate(
    mask, size=(side, side),
    mode="bilinear", align_corners=False
).squeeze()

if threshold > 0:
    mask = (mask > threshold).float()

mask_np = (mask.clamp(0, 1).cpu().numpy() * 255).astype(np.uint8)
mask_cropped = mask_np[pad_y:pad_y + h, pad_x:pad_x + w]
mask_pil = Image.fromarray(mask_cropped, mode="L")

```

```

    rgba = rgb_image.convert("RGBA")

    rgba.putalpha(mask_pil)

    return rgba

```

Б.4 QC-гейты конвейера пакшота

```

def construct_qc_gates(
    original_np_rgb, final_mask_np, bbox,
    canvas=None, full_canvas_mask=None,
    margin_ratio=0.05,
) -> dict:
    """Evaluate pipeline quality based on strict gates."""
    h, w = original_np_rgb.shape[:2]
    x, y, w_b, h_b = bbox
    gates = {}
    failed_reasons = []

    # G1: Spatial Bounds
    if (x <= 1 or y <= 1
        or (x + w_b) >= w - 1 or (y + h_b) >= h - 1):
        failed_reasons.append("SOURCE_CROPPED")
        gates["G1_SPATIAL_BOUNDS"] = "FAILED"
    else:
        gates["G1_SPATIAL_BOUNDS"] = "PASSED"

    # G4: Integrity
    contours, _ = cv2.findContours(
        final_mask_np, cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    major = [c for c in contours if cv2.contourArea(c) > 500]

```



```

if len(major) > 3:
    failed_reasons.append("FRAGMENTED_MASK")

if canvas is not None and full_canvas_mask is not None:
    # G2: Background Purity
    bg_mask = (full_canvas_mask == 0)
    bg_pixels = canvas[bg_mask]
    if len(bg_pixels) > 0:
        white_cond = (
            (bg_pixels[:, 0] >= 248)
            & (bg_pixels[:, 1] >= 248)
            & (bg_pixels[:, 2] >= 248)
        )
        ratio = np.sum(white_cond) / len(bg_pixels)
        if ratio < 0.98:
            failed_reasons.append("DIRTY_BACKGROUND")

    # G3: Halo Outer Ring
    kernel = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (11, 11)
    )
    dilated = cv2.dilate(full_canvas_mask, kernel)
    outer = cv2.bitwise_xor(dilated, full_canvas_mask)
    ring_pixels = canvas[outer > 0]
    if len(ring_pixels) > 0:
        white_cond = (
            (ring_pixels[:, 0] >= 248)
            & (ring_pixels[:, 1] >= 248)
            & (ring_pixels[:, 2] >= 248)
        )

```

```

        outer_ratio = np.sum(white_cond) / len(ring_pixels)
        if outer_ratio < 0.95:
            failed_reasons.append("HALO_DETECTED")

    status = "FAILED_NEEDS_REVIEW" if failed_reasons else "PASSED"
    return {
        "status": status,
        "gates": gates,
        "failed_reasons": list(dict.fromkeys(failed_reasons)),
    }

```

Б.5 Frontend hook для SSE-прогресса

```

import { useEffect, useRef, useState } from 'react';

export interface BatchProgressEvent {
    type: 'snapshot' | 'job_started' | 'item_started'
        | 'item_stage' | 'item_completed' | 'job_completed';
    total?: number;
    item_index?: number;
    filename?: string;
    stage?: string;
    pct?: number;
    image_url?: string | null;
}

```

```

export interface BatchProgressState {
    total: number;
    completed: number;
    currentItemIndex: number;
    currentFilename: string;
    currentStage: string;
}

```

```

    currentPct: number;
    isDone: boolean;
  }

export function useBatchProgress(
  jobId: string | null
): BatchProgressState {
  const [state, setState] = useState<BatchProgressState>(initial);
  const esRef = useRef<EventSource | null>(null);

  useEffect(() => {
    if (!jobId) {
      setState(initial);
      return;
    }

    const base = (
      process.env.NEXT_PUBLIC_API_URL
      || 'http://localhost:8000/api/v1'
    ).replace(/\\/api\\/v1\\/?$/, '');
    const url = `${base}/api/v1/ai/batch_jobs/${jobId}/stream`;
    const es = new EventSource(url);
    esRef.current = es;

    es.onmessage = (raw) => {
      try {
        const event: BatchProgressEvent = JSON.parse(raw.data);
        setState((prev) => reduce(prev, event));
      } catch {}
    };
  });

```

```
    return () => {  
        es.close();  
        esRef.current = null;  
    };  
    }, [jobId]));  
  
    return state;  
}
```